Final Technical Report
Whisper Notes

Introduction

Whisper Notes is a React application for creating structured notes from uploaded or recorded audio. The current implementation connects the main views into a realistic workflow: a user signs in, submits audio for transcription, reviews the generated result, and can share it with all other users. The interface follows the green visual style introduced after the original design draft and includes responsive layouts for smaller screens.

Implemented Functionality

- **Application flow and navigation:** The application uses `react-router-dom` for the Login, Home, Transcribe, Results, and Shared Files views. Protected routes redirect unauthenticated users to `/login`. After login, the user is redirected to Home and can navigate between the four main application views.
- **Login and Home:** The prototype Login view now requires only a name or username. The unused password field and inactive THWS Login button were removed to make the current behavior clear and user-friendly. The active user is stored in browser `localStorage`, which keeps the session after a refresh. Private transcription results are stored under normalized, username-specific keys, so a user can log out, sign in again with the same username, and access the same locally stored results. The Home view displays the user profile and provides direct links to the main workflows.
- **Audio input and transcription:** The Transcribe view accepts common audio formats through upload, drag and drop, or a keyboard-accessible click-to-browse dropzone. Selected files are validated against the supported formats and a 200 MB size limit, and the file summary shows the name, size, and duration read from the audio metadata. Audio can also be recorded directly from the microphone with the browser `MediaRecorder` API; recordings receive an automatic date-based title. Users can enter a title, select a language, enable timestamps, and choose a transcription engine. Engine configuration (excluding API keys), language, and timestamp preferences persist in `localStorage` between sessions. A running transcription can be cancelled through an `AbortController`, failed requests offer a retry action, the form is locked while a request is active, and microphone streams, temporary object URLs, and open requests are released when the view is left.
- **WhisperLive integration:** The previous simulated progress flow was replaced with asynchronous communication with the OpenAI-compatible WhisperLive REST endpoint. Audio is sent as multipart form data to `/v1/audio/transcriptions`. The interface displays an indeterminate loading state, handles server errors, converts successful responses into the application's result structure, and links directly to the generated result.
- **Results:** Generated results are stored per normalized username in `localStorage`. The Results view only displays entries belonging to the active user. It supports audio playback, an interactive progress control, timestamp-based transcript highlighting, speaker-name editing, transcript copying, and text download. Results can also be shared through a dedicated Share action.
- **Shared Files:** Shared Files is common to all users. It combines the existing sample entries with asynchronously loaded shared records. Users can select a file, play its audio, seek through the interactive progress bar, and follow the transcript segment corresponding to the current playback time.
- **Shared-data persistence:** A small Node.js HTTP service provides health, list, share, and audio-serving endpoints. Shared transcript metadata is written to `docs/shared-transcriptions.json`, while uploaded shared audio is stored under `docs/audio/shared/`. Vite proxies `/api` requests to this service during local development.
- **Local development and Docker:** The default development command starts Vite and the persistence service together. Docker Compose provides a complete stack containing the production React build served by Nginx, the Node.js persistence service, and the WhisperLive CPU service.

Health checks, an Nginx API reverse proxy, a bind mount for shared data, and a named volume for downloaded Whisper models are included.

Recent Frontend Improvements

- The login flow was simplified to accept only a name or username during the prototype stage. Empty usernames are rejected with a clear validation message.
- The unused password input and non-functional THWS Login button were removed.
- The Login page was redesigned with a clearer visual hierarchy, improved input focus states, concise guidance, and a more prominent call-to-action button.
- The Home greeting was changed from the informal "Hi, username" format to a cleaner personalized workspace header without the comma.
- A direct "Start transcribing" action was added to the Home welcome section to reduce the number of steps needed to begin a transcription.
- The Home welcome panel, avatar, spacing, shadows, and responsive styling were refined. An unintended white background layer behind and between the dashboard cards was removed to create a cleaner page background.
- The Transcribe page was completed to its final state: uploaded and dropped files are validated for supported format and a 200 MB size limit, the selected file's duration is read from its metadata and shown in the file summary, and the drag-and-drop zone was made clickable and keyboard accessible.
- Engine, language, and timestamp preferences on the Transcribe page now persist across sessions through `localStorage`, while API keys are intentionally never stored.
- Transcription requests can now be cancelled while running and retried after a failure, the whole form is disabled during an active request, recordings receive an automatic date-based title, and microphone streams, temporary object URLs, and pending requests are released when the user leaves the page.
- The frontend changes were verified with a successful Vite production build.

Major Technical Decisions

- React and Vite were retained to match the existing frontend structure and provide a lightweight development and production build process.
- WhisperLive remains an external service instead of being copied into this repository. The frontend communicates with its OpenAI-compatible REST API, which keeps the speech-processing implementation separate from the user interface.
- Username normalization is used to scope private results without introducing unfinished authentication logic. Shared records intentionally remain global because the Shared Files view represents a common workspace.
- Private prototype results use browser storage, while shared records use a minimal file-based application API. This provides demonstrable persistence and asynchronous communication without introducing a database before the final backend architecture is defined.
- Nginx serves the compiled single-page application and forwards `/api` traffic to the persistence service. Direct URLs such as `/results` continue to work through the SPA fallback.
- The Docker configuration uses the available `linux/amd64` WhisperLive CPU image. It runs natively on x86-64 systems and through Docker emulation on ARM-based systems such as Apple Silicon.

Remaining Limitations

- Login is still a prototype and username entry is not real authentication. Any person who enters the same username on the same device can access that username's locally stored results.
- Home dashboard totals are currently presentation data and are not calculated from the active user's stored results.

- Private result metadata is limited to recent browser-stored entries. The associated temporary audio URL is only available during the current application session unless the result is shared.
- The persistence service uses JSON files rather than a database and does not yet provide accounts, permissions, deletion, search, or production-level concurrent access.
- REST speaker diarization is disabled because the current WhisperLive flow requires additional reference-audio handling. Live WebSocket transcription is also not yet exposed in the interface.
- Cloud provider keys are sent directly from the browser. A secure server-side proxy is required before this option can be considered production ready.
- WhisperLive currently runs with the CPU image. On Apple Silicon, the required AMD64 emulation can make model loading and transcription slower, especially during the first run.

Authentication and Database Requirements

The current username-based flow is suitable for demonstrating the frontend, but it does not provide identity verification or secure access control. Converting it into a real authentication system requires coordinated backend and database development. The backend must provide registration, login, logout, and current-user endpoints. User records must be stored in a database with a unique user ID and unique username or email address. If application passwords are used, they must never be stored as plain text; they must be salted and hashed with a suitable password-hashing algorithm such as bcrypt or Argon2.

After successful login, the application should use a secure server-side session or a carefully implemented token-based mechanism. Protected backend endpoints must verify the authenticated user before returning or modifying data. Transcriptions, audio files, and shared records should reference the user's database ID rather than relying on a username supplied by the browser. Authorization rules must ensure that private results are visible only to their owner, while deliberately shared files remain available to the intended users.

To connect the frontend to this system, the backend team must define the database technology, data model, API routes, request and response formats, session or token strategy, and development environment configuration. Database credentials and authentication secrets must be supplied through environment variables and must not be committed to Git. Once this contract is available, the current frontend `localStorage` login can be replaced with API-based authentication and database-backed user data.

Next Steps

The next development stage should introduce real THWS or application authentication, database-backed storage for private and shared results, persistent audio management, and authorization rules. Further work should also add secure cloud-provider integration, speaker diarization, live transcription, dynamic Home statistics, automated API and component tests, and additional shared-file management features.

Codex/AI tools were used to support implementation planning, code generation and review, technical troubleshooting, grammar improvement, and the preparation of this LaTeX document.